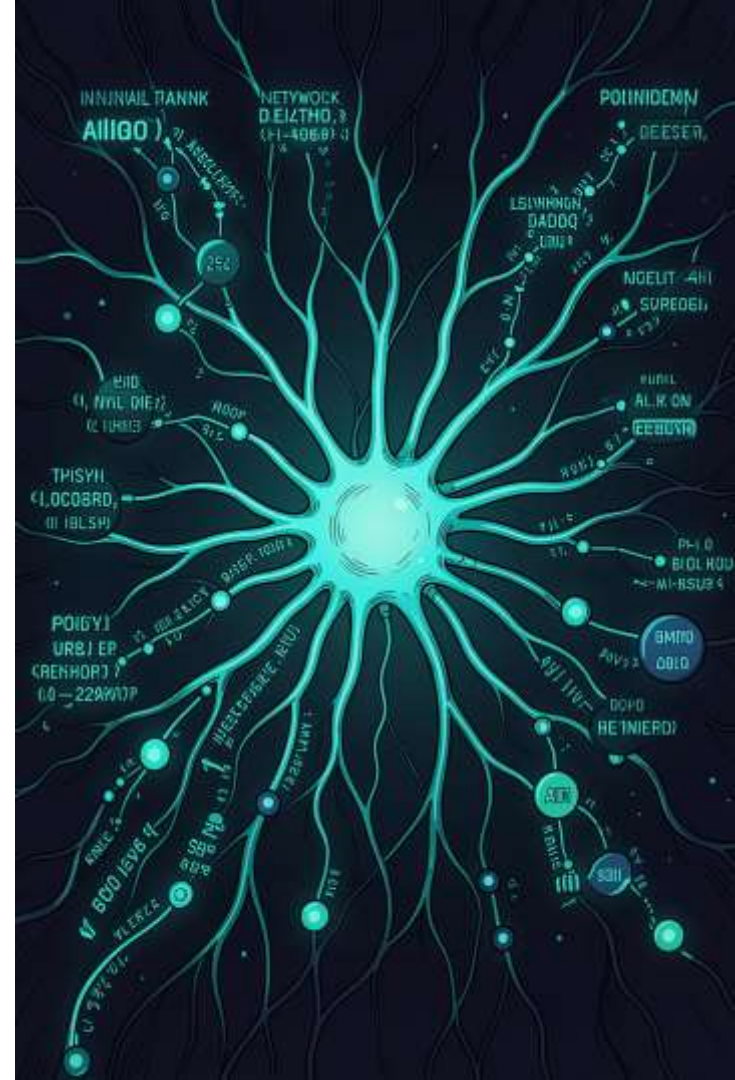


AI-Powered Fake News Detection System

A Desktop Application for Real-Time Text Classification

This system uses **Natural Language Processing (NLP)** and **Machine Learning** to evaluate linguistic patterns and calculate truth probabilities — rather than relying on a static fact-checking database.





The Challenge of Online Misinformation

Information Overload

Deceptive articles, clickbait, and altered narratives spread faster than manual verification can keep up.

Static Database Limits

Existing fact-checking tools rely on fixed databases — completely ineffective against newly written or emerging fake news.

Linguistic Deception

Fabricated news uses sensationalized structures and vocabulary patterns that are invisible to humans but ideal for algorithmic analysis.

Project Repository Structure

FAKE_NEWS_DETECTION

docs

Algorithm.txt

Flowchart.txt

include

__init__.py

database.py

text_processing.py

src

app.py

ml_pipeline.py

.gitignore

main.py

requirements.txt

include/

database.py (MongoDB logic) and
text_processing.py (regex text cleaning)

src/

app.py (Tkinter UI) and ml_pipeline.py (ML
model logic)

docs/

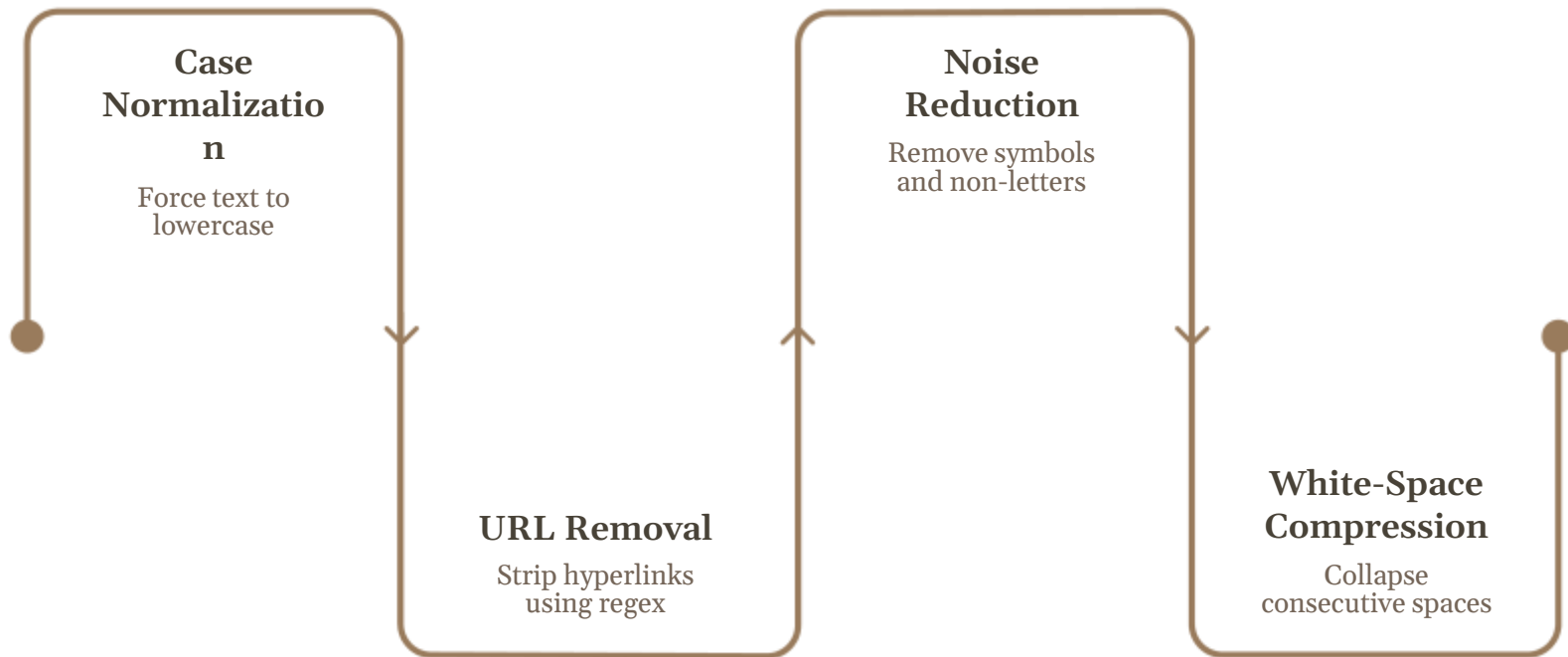
System documentation: Algorithm.txt and
Flowchart.txt

Root

Entry point via main.py; dependencies
managed in requirements.txt

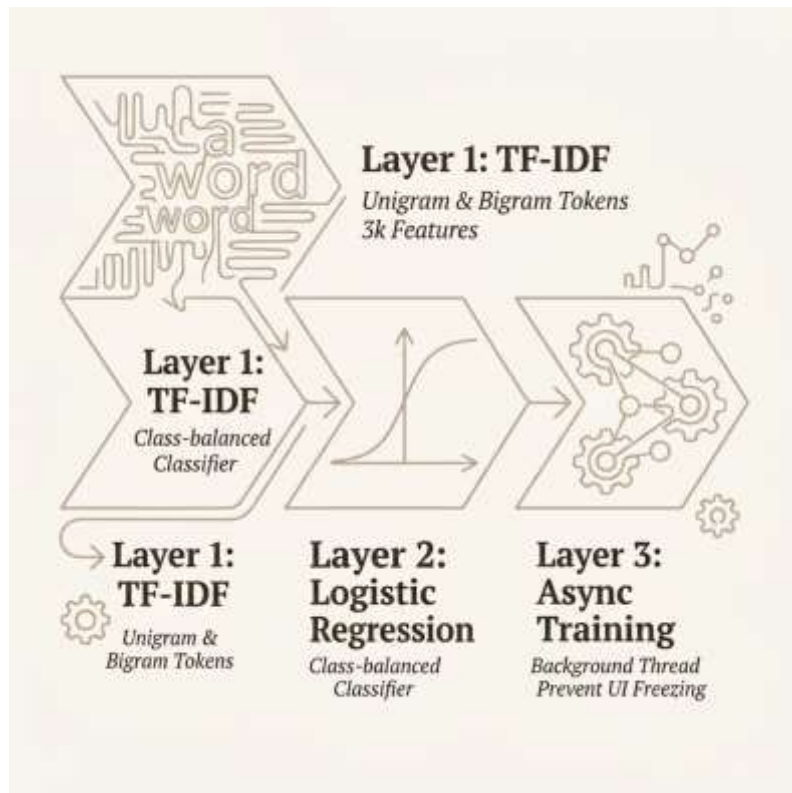
Data Cleaning & Normalization

Raw user input passes through a sequential preprocessing pipeline before any model inference occurs.



Each step eliminates a distinct class of noise, producing a clean, normalized token stream optimized for TF-IDF vectorization.

Feature Extraction & Model Fitting



TF-IDF Vectorizer

Extracts unigram and bigram token frequencies, capped at **3,000 vocabulary features** to balance expressiveness and performance.

Logistic Regression Classifier

A **class-balanced** model trained to handle imbalanced real/fake news datasets without bias toward the majority class.

Asynchronous Training

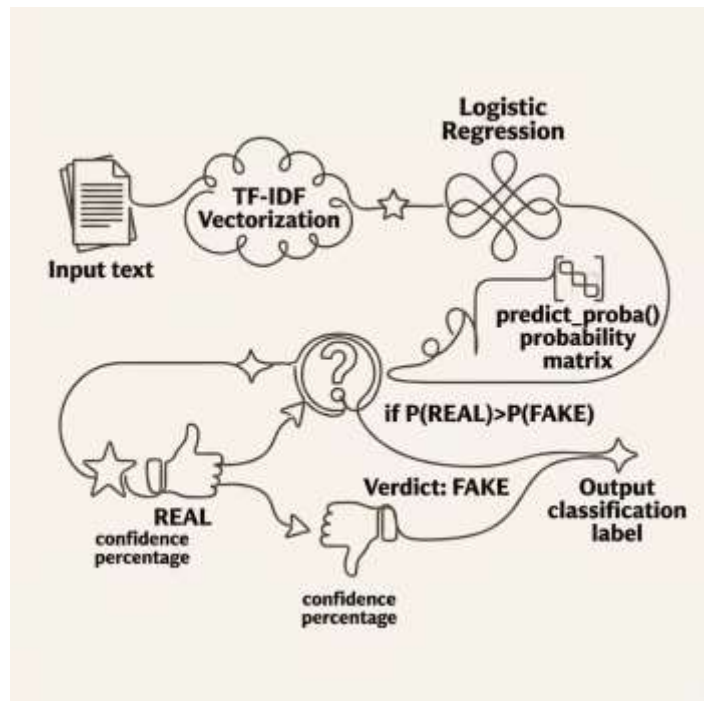
Model fitting runs on a **separate background thread** — the UI never freezes or stutters during training or prediction.

Decision-Making Flow & Classification

Probability-Based Verdict

Rather than a blind label, the system queries `.predict_proba()` to extract explicit probability matrices for both classes.

- Output includes a **definitive classification label** alongside a **percentage confidence score** for full transparency.



✓ REAL

$P(\text{REAL}) > P(\text{FAKE})$

☹ FAKE

$P(\text{FAKE}) \geq P(\text{REAL})$

Intelligent Database Integration



Online Mode

If a local MongoDB cluster is detected, the app auto-connects and logs transaction history **asynchronously** in the background.



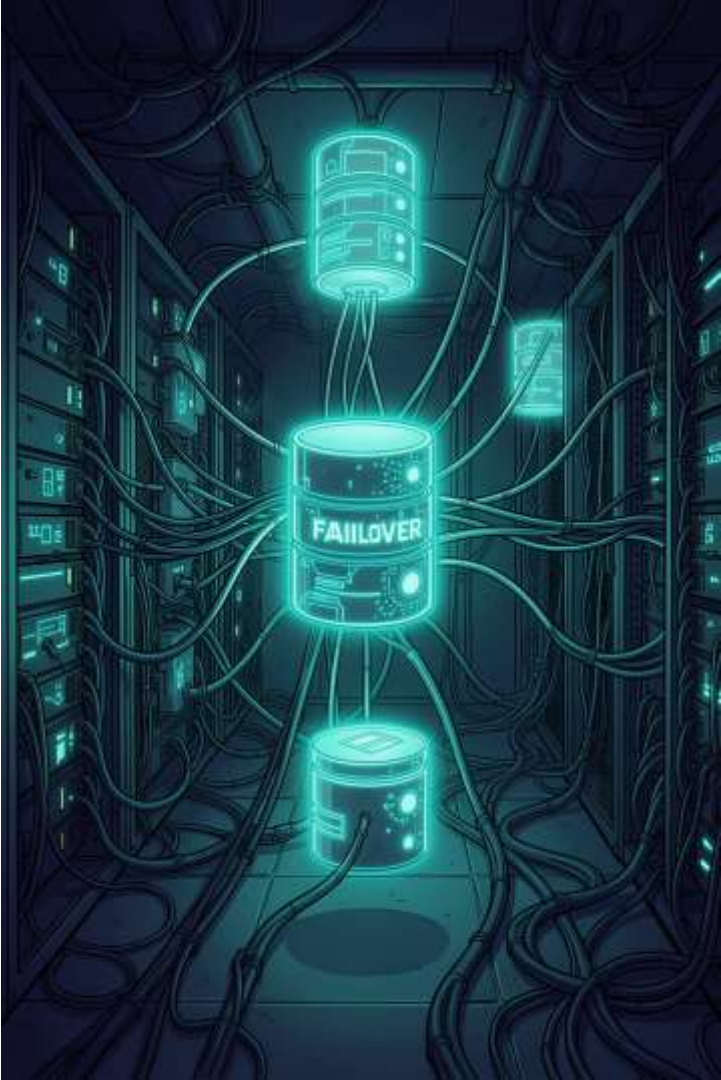
Offline Mode

If the database is unavailable, the system **seamlessly bypasses** the connection and runs entirely in local volatile memory.



100% Uptime

Guarantees full operational functionality in either state — **no crashes, no errors**, no interrupted sessions.



Graphical UI & Interactive Dashboards



Intuitive Layout

Built with **Tkinter**, featuring configuration spinboxes for dataset splits (Train / Val / Test).



Color-Coded Feedback

Results render instantly on visual cards — **bright green** for Real news patterns, **bold red** for Fake news flags.



Session Analytics

A **Matplotlib-powered** popup dashboard renders interactive pie charts and historical prediction metrics.

Roadmap & Next-Gen Enhancements



Advanced NLP Architectures

Upgrade from Logistic Regression to **Transformers or BERT** for deeper contextual text understanding.



Real-Time Web Scraping

Integrate live API connections or web scrapers to **fetch and analyze online links** directly from the interface.



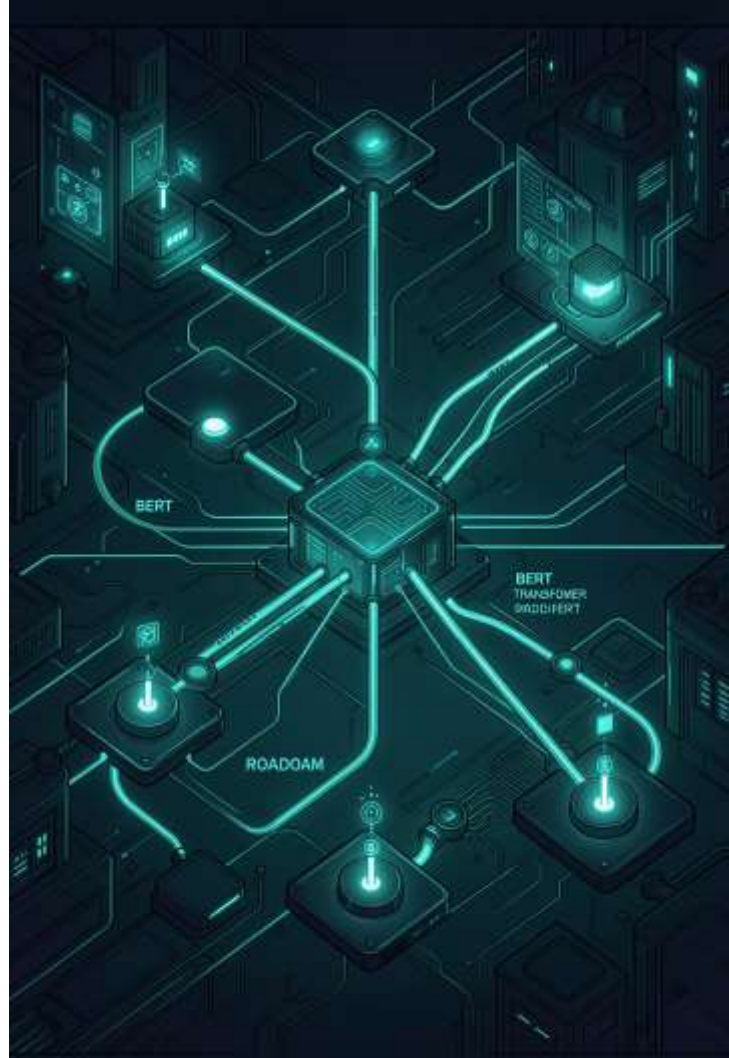
Cross-Lingual Capabilities

Extend the regex pipeline and vectorizer tokens to **process and verify** articles in multiple languages.



Browser Extension

Port the core classification logic into a **lightweight browser extension** for on-page news verification.



Summary & Engineering Highlights

3

Modular Packages

UI, NLP, and database layers separated
for clean maintainability

Modular Design

Clear separation of concerns across
`include/`, `src/`, and `docs/` packages.

100%

UI Responsiveness

Threaded background processing keeps
the interface completely responsive

⚡ User Experience First

Asynchronous threading ensures the
UI never blocks during model
training or database I/O.

2

Database Modes

Online MongoDB + offline volatile
memory fallback for zero-downtime
operation

☑ High Reliability

Graceful database fallbacks
guarantee software durability across
all runtime environments.